

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
<i>Desenvolvimento de Software para Dispositivos Móveis</i>	
Nome do(a) acadêmico(a)	Valor 7,0
Professor <i>Sergio Souza Novak</i>	Turma
Data da Avaliação 02/10/2026	Código ESW417

ATENÇÃO: Somente serão passíveis de **REVISÃO** avaliações resolvidas à **TINTA**.

ORIENTAÇÕES PARA A RESOLUÇÃO

- A avaliação é individual, sem consulta;
- A interpretação de texto faz parte da avaliação;

QUESTÕES

Código fonte de apoio para as questões 1 até 4

```
package com.sergiosouzanovak.corpodocenteunirv

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CorpoDocenteUnirvTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    App(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun carregarImagemAsset(nomeArquivo: String): ImageBitmap? {
    val context = LocalContext.current

    return try {
        context.assets.open(nomeArquivo).use {
            inputStream ->
            BitmapFactory.decodeStream(inputStream)?.asImageBitmap()
        }
    } catch (e: Exception) {
        null
    }
}
```

```
}
} catch (e: Exception) {
    null
}
}

@Composable
fun PesquisadorCard(
    nome: String,
    titulacao: String,
    lattes: String,
    email: String,
    foto: String,
    modifier: Modifier = Modifier
) {
    val imagem = carregarImagemAsset(foto)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .padding(16.dp),
        shape = RoundedCornerShape(4.dp),
        colors = CardDefaults.cardColors(
            containerColor = Color(0xFFFF1F4F5)
        ),
        elevation = CardDefaults.cardElevation(
            defaultElevation = 0.dp
        )
    ) {
        Column(
            modifier = Modifier.width(1200.dp).height(300.dp),
            horizontalAlignment = Alignment.
```

```

        CenterHorizontally
    ) {
        imagem?.let {
            Image(
                bitmap = it,
                contentDescription = "Foto de $nome",
                modifier = Modifier
                    .fillMaxWidth()
                    .height(120.dp),
                contentScale = ContentScale.Crop,
            )
        }
        Text(
            text = nome,
            modifier = Modifier.padding(
                top = 14.dp,
                start = 8.dp,
                end = 8.dp
            ),
            fontSize = 12.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF222222),
            textAlign = TextAlign.Center
        )
        Text(
            text = titulacao,
            modifier = Modifier.padding(top = 6.dp),
            fontSize = 12.sp,
            color = Color(0xFF666666),
            textAlign = TextAlign.Center
        )
        Text(
            text = lattes,
            modifier = Modifier
                .padding(
                    top = 28.dp,
                    start = 16.dp,
                    end = 16.dp
                )
                .clickable {
                    // abrir Lattes
                },
            fontSize = 12.sp,
            color = Color(0xFF007AFF),
            textAlign = TextAlign.Center
        )
        Text(
            text = email,
            modifier = Modifier.padding(
                top = 14.dp,
                bottom = 22.dp
            ),
            fontSize = 12.sp,
            color = Color(0xFF444444),
            textAlign = TextAlign.Center
        )
    }
}

data class Docente(
    val nome: String,
    val titulacao: String,
    val lattes: String,
    val email: String,
    val foto: String
)

```

```

@Composable
fun ListaPesquisadores() {
    val docentes = listOf(
        Docente(
            nome = "ALEX MIZAEI MARTINS",
            titulacao = "Doutor",
            lattes = "www.google.com",
            email = "sergio.novak@unirv.edu.br",
            foto = "docentes/ALEX MIZAEI MARTINS.png"
        ),
        Docente(
            nome = "BACUS DE OLIVEIRA NAHIME",
            titulacao = "Doutor",
            lattes = "www.google.com",
            email = "joao@unirv.edu.br",
            foto = "docentes/BACUS DE OLIVEIRA NAHIME.png"
        ),
        Docente(
            nome = "Barbara Gomes Martins",
            titulacao = "Mestre",
            lattes = "www.google.com",
            email = "maria@unirv.edu.br",
            foto = "docentes/Barbara Gomes Martins.png"
        ),
        Docente(
            nome = "BIANCA SIMONELI DE OLIVEIRA",
            titulacao = "Doutor",
            lattes = "www.google.com",
            email = "pedro@unirv.edu.br",
            foto = "docentes/BIANCA SIMONELI DE OLIVEIRA.png"
        )
    )

    LazyVerticalGrid(
        columns = GridCells.Fixed(2),
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        items(docentes) { docente ->
            PesquisadorCard(
                nome = docente.nome,
                titulacao = docente.titulacao,
                lattes = docente.lattes,
                email = docente.email,
                foto = docente.foto
            )
        }
    }
}

@Composable
fun PageTitle() {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Row {
            Text(text = "Corpo Docente do curso de")
            Text(
                text = " Engenharia Civil",
                color = Color(0xFF064F9F)
            )
        }
        Row {
            Column {
                repeat(2) {
                    HorizontalDivider(

```

<pre> modifier = Modifier.width (125.dp), thickness = 2.dp, color = Color.Gray) Spacer(modifier = Modifier. height(4.dp)) } } Row{ ListaPesquisadores() } } </pre>	<pre> @Composable fun App(modifier: Modifier) { Column{ PageTitle() } } @Preview(showBackground = true) @Composable fun GreetingPreview() { CorpoDocenteUnirvTheme { App(modifier = Modifier) } } </pre>
---	---

Questão 1 - No Jetpack Compose, a anotação `@Composable` é fundamental para a construção de interfaces de usuário. Qual é a principal função dessa anotação (utilizada, por exemplo, nas funções `PesquisadorCard` e `ListaPesquisadores`) no código fornecido?

- a) Indicar ao compilador que a função retorna uma visualização tradicional baseada em arquivos XML do Android.
- b) Transformar a função em um componente de interface gráfica que o framework do Compose pode processar para desenhar a tela.
- c) Configurar a comunicação direta entre o aplicativo e o banco de dados interno do dispositivo móvel.
- d) Declarar que a função correspondente deve ser executada de forma assíncrona, visando otimizar a rede.
- e) Gerenciar o ciclo de vida da Activity principal (MainActivity) substituindo automaticamente o método `onCreate`.

Questão 2 - Sobre os componentes de layout `Row` e `Column`, amplamente utilizados no Jetpack Compose (como visto na função `PageTitle`), julgue as afirmativas abaixo como Verdadeiras (V) ou Falsas (F):

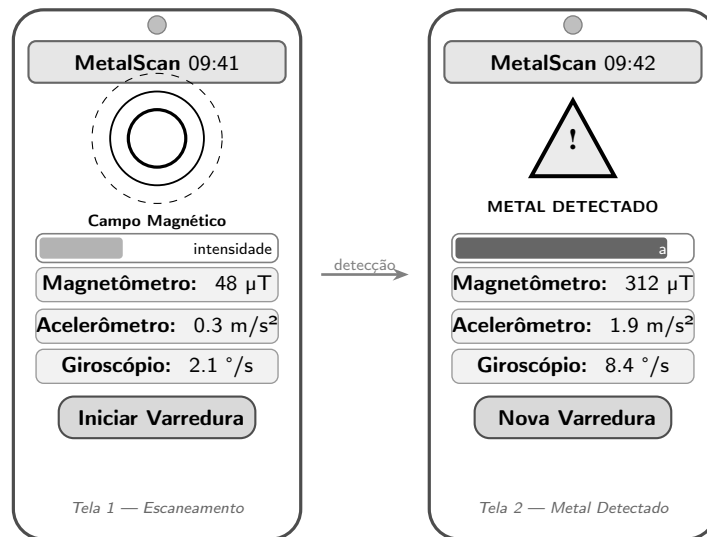
- () O componente `Column` organiza seus elementos filhos sequencialmente na vertical (um abaixo do outro).
- () O componente `Row` organiza seus elementos filhos sequencialmente na horizontal (um ao lado do outro).
- () Em uma `Column`, para alinhar os elementos filhos no eixo horizontal (como esquerda, centro ou direita), utiliza-se o parâmetro `horizontalAlignment`.

Questão 3 - No desenvolvimento Android, observa-se o uso constante de unidades como `dp` (Density-independent Pixels) para as dimensões dos componentes (ex: `16.dp`) e `sp` (Scale-independent Pixels) para o tamanho das fontes (ex: `12.sp`). Explique qual é o problema de se utilizar a medida absoluta de pixels (`px`) na construção de telas e quais são as vantagens de se adotar o `dp` e o `sp`.

Questão 4 - No código de apoio, observamos duas estruturas responsáveis por exibir componentes de interface: a função `GreetingPreview()` (anotada com `@Preview`) e a classe `MainActivity` (que herda de `ComponentActivity`). Explique a principal diferença entre os dois trechos de código. Em sua resposta, indique claramente qual deles é utilizado apenas durante a fase de desenvolvimento (para pré-visualização na IDE) e qual deles é efetivamente invocado pela aplicação real quando executada no dispositivo móvel.

Questão 5 - Uma equipe de desenvolvimento deseja criar um aplicativo para smartphones capaz de auxiliar na detecção de metais utilizando os sensores já disponíveis no dispositivo móvel. Durante o projeto, foram identificados os seguintes sensores:

- Um sensor que mede a intensidade do campo magnético ao redor do celular.
- Um sensor que detecta movimentos e acelerações do aparelho.
- Um sensor que identifica a orientação e a rotação do dispositivo.



Considerando o funcionamento desse aplicativo para detecção de metais, assinale a alternativa que indica corretamente qual é o principal sensor responsável por medir as variações e identificar a presença do metal ao redor do aparelho:

- a) Acelerômetro
- b) Magnetômetro
- c) Giroscópio
- d) Sensor de Proximidade
- e) Barômetro

Questão 6 - Analise o seguinte trecho de código escrito em Kotlin:

```
val soma = { a: Int, b: Int -> a + b }

fun main() {
    println(soma(5, 3)) // Saída: 8
}
```

O trecho de código acima demonstra uma importante característica da linguagem Kotlin. Assinale a alternativa que descreve corretamente o conceito utilizado na declaração da variável `soma`:

- a) Tratamento de exceções utilizando blocos anônimos para capturar erros matemáticos.
- b) Declaração de uma função de extensão estática na classe `Int`.
- c) Instanciação de uma classe abstrata responsável pelas operações aritméticas básicas.
- d) Definição de uma função lambda (função anônima), que é tratada como um tipo de primeira classe e pode ser armazenada em variáveis.
- e) Sobrecarga do operador de adição nativo da linguagem para modificar seu funcionamento.

Questão 7 - Observe o trecho de código Kotlin abaixo, que apresenta diferentes formas de inicializar coleções de dados:

```
val lista = mutableListOf("Item 1")
val lista2 = arrayListOf("Item 1") // nativo

// Set (HashSet)
val conjunto = mutableSetOf(10)
val conjunto2 = hashSetOf(10)

// Map (HashMap)
val mapa = mutableMapOf("Chave" to 100)
val mapa2 = hashMapOf("Chave" to 100)
```

A linguagem Kotlin provê diversas funções utilitárias para a criação de coleções. Com base nas estruturas apresentadas (`List`, `Set` e `Map`) e nos conceitos de coleções em Kotlin, assinale a afirmativa correta:

- a) As funções que possuem o prefixo `mutable` criam coleções de leitura restrita (imutáveis), cujos elementos não podem ser alterados após a instanciação.
- b) A estrutura `Set` (como a criada por `mutableSetOf` ou `hashSetOf`) é projetada para permitir o armazenamento de vários elementos idênticos e duplicados.
- c) A estrutura `Map` armazena elementos utilizando o paradigma de associação chave-valor, onde cada chave mapeada deve ser necessariamente única na coleção.
- d) O uso da palavra-chave `val` na declaração das variáveis impede que novos elementos sejam adicionados ou removidos das listas e conjuntos mutáveis.
- e) Em Kotlin, não existe diferença conceitual entre `List` e `Set`, pois ambas utilizam índices inteiros, não possuem ordem e não permitem repetidos.

Questão 8 - A linguagem Kotlin oferece diversas funções de alta ordem (higher-order functions) que facilitam a manipulação e transformação de coleções de forma funcional, sendo `filter` e `map` algumas das mais utilizadas. Sobre o tema, julgue as afirmativas abaixo como Verdadeiras (V) ou Falsas (F):

- () A função `filter` é utilizada para percorrer uma coleção e retornar uma nova coleção contendo apenas os elementos que satisfazem uma dada condição (predicado).
- () A função `map` tem o objetivo de transformar os itens de uma coleção, aplicando uma função fornecida a cada elemento original e retornando uma nova coleção com os resultados gerados.
- () O uso das funções `filter` e `map` altera e substitui diretamente os valores dentro da coleção original na memória, não sendo necessário armazenar o retorno.
- () Ao utilizar expressões lambdas no Kotlin (como dentro do bloco do `filter` ou `map`), o identificador implícito `it` pode ser utilizado para referenciar o elemento atual da iteração, encurtando o código.